Genetic Prompt Search for JOILang Code Generation under Deterministic Validation and Operational Feedback

Mingi Jeong[1,2], Eunjin Jeong[2], and Soonhoi Ha[2]
[1]Electronics and Telecommunications Research Institute (ETRI), Daejeon, Republic of Korea
[2]Department of Computer Science and Engineering, Seoul National University, Seoul, Republic of Korea

Abstract—기존 JOILang 계열 연구는 서비스 수준 IoT DSL로서의 JOILang, 신뢰성 중심의 NL-to-JOILang 번역 파이프라인, 그리고 서비스 지향 IoT 실행 인프라를 핵심 기여로 확립하였다. 본 논문은 이 축을 반복하지 않고, 빠르게 등장하는 local 및 cloud 모델 환경에서 JOILang 코드 생성을 위한 프롬프트를 버전 관리 가능한 소프트웨어 아티팩트로 재정의한다. 구체적으로, 본 논문은 프롬프트를 block-structured genotype으로 표현하고, benchmark 성능, 운영 실패 replay, 결정적 검증 신호, 선택적 semantic judge를 결합한 유전 알고리즘 기반 탐색 루프를 제안한다. 또한 prompt 길이, latency, 회귀 위험을 함께 고려하는 multi-objective fitness와 regression-aware acceptance gate를 통해 정확도 향상과 입력 토큰 감소를 동시에 추구한다. 실험은 GPT-4.1-mini를 상위 기준선으로 두고, Phi-3.5-mini-instruct, Qwen2.5-Coder-7B-Instruct, Llama-3.1-8B-Instruct, Gemma-2-9B-it, Qwen2.5-Coder-14B-Instruct로 구성된 size-tiered representative local model suite에서 수행되며, JOILang 중심 평가가 방법론 일반성 주장으로 과도하게 해석되지 않도록 schema-constrained auxiliary structured generation task를 추가로 포함한다. 그 결과, cloud-authored prompt를 local 모델에 직접 전이하는 방식보다 제안 방식이 평균 DETPass를 5.9%p, ReplayAcc를 13.1%p 향상시키면서도 입력 토큰 수를 평균 66.7% 감소시키는 파레토 개선을 보였다. 이러한 결과는 JOILang generation을 위한 prompt engineering을 수작업 문구 조정이 아니라 search, evaluation, acceptance, deployment로 이어지는 PromptOps lifecycle 문제로 다루어야 함을 시사한다.

Index Terms—genetic prompt search, PromptOps, deterministic evaluation, local LLMs, evolutionary prompt optimization, JOILang, Skeleton-of-Thought

1 서론

최근 거대 언어 모델(LLM)을 활용한 IoT 제어 자동화에서 성능은 프롬프트 설계에 크게 의존한다. 기존 연구들은 JOILang 이라는 서비스 수준의 특화 언어(DSL)와 Service Pre-mapping 기반의 검색 증강 프롬프트, 그리고 결정적 검증 계층을 결합하여 신뢰성 중심의 NL-to-JOILang 파이프라인을 확립하였다. 클라우드 기반의 최신 모델, 예를 들어 GPT-4.1-mini와 같은 계열은 장황한 지시문과 퓨샷 예시를 통해 높은 정확도를 달성할 수 있다. 그러나 엣지(Edge) 환경이나 온프레미스(On-premise)로 운영되는 폐쇄망 IoT 시스템에서는 클라우드 API 의존성, 높은 토큰 처리 비용, 그리고 지연 시간(latency)이 심각한 병목으로 작용한다.

이를 해결하기 위해 Qwen, Llama, Gemma, Phi 계열과 같은 경량 Local LLM을 도입하는 것이 자연스럽지만, 실제 운영에서는 또 다른 문제가 발생한다. 클라우드 모델을 위해 작성된 프롬프트를 local 모델에 그대로 이식하면 문법 붕괴, 스키마 위반, 환각, 그리고 긴 문맥 처리 실패가 자주 발생한다. 가중치 미세조정(weight fine-tuning)은 서비스 스키마 변경, 디바이스 추가, 운영 정책 수정이 잦은 환경에 대응하기에는 비용이 높고 운영 부담이 크다. 이 때문에 local/deployment-oriented 환경에서는 모델 파라미터를 바꾸기보다, 프롬프트와 설정 자체를 빠르게 탐색하고 교체할 수 있는 artifact-level optimization이 훨씬 실용적이다.

본 논문은 이러한 맥락에서 프롬프트 엔지니어링을 일회성 문구 조정이 아니라 **소프트웨어 아티팩트 최적화(artifact optimization)** 문제로 재정의한다. 구체적으로, 클라우드 모델(GPT-4.1-mini)의 성능을 상위 기준선으로 두고, local LLM이 이를 토큰 효율적으로 따라잡을 수 있도록 프롬프트를 기능적 단위의 블록(block-structured prompt)으로 분해한 뒤, 이를 유전 알고리즘(GA) 기반으로 탐색한다. 탐색 과정은 benchmark 정답 성능뿐 아니라 운영 실패 replay, 결정적 검증 신호, 토큰 길이, latency, 회귀 위험을 함께 고려하는 다목적 적합도(multi-objective fitness)로 구성된다.

중요한 점은 본 논문이 JOILang 자체를 새로 제안하는 것이 아니라는 점이다. JOILang, 기존 reliability-oriented 파이프라인, 그리고 서비스 지향 IoT 실행 인프라는 이미 foundation으로 존재한다. 본 논문의 실제 novelty는 다음 세 방향에 있다. 첫째, JOILang generation prompt를 versioned artifact 이자 searchable genotype으로 재정의한다. 둘째, benchmark와 replay, deterministic validation을 search-time fitness로 통합한 evolutionary PromptOps 루프를 설계한다. 셋째, local model family별로 cloud-authored prompt의 직접 전이가 충분하지 않음을 보이고, model-specific prompt adaptation이 accuracy와 token efficiency를 동시에 개선함을 입증한다. 또한 JOILang 내부 성능 향상만으로 방법론의 객관적 기여를 과장하지 않기 위해, 본 논문은 auxiliary structured generation task를 추가하여 block-level search와 deterministic fitness의 전이 가능성을 제한적으로 점검한다.

본 연구의 핵심 기여는 다음과 같다.

- JOILang generation prompt를 block-structured genotype으로 재정의하고, prompt artifact 자체를 GA/evolutionary search의 대상으로 정식화한다.
- 정확도 향상뿐 아니라 토큰 압축, 결정적 검증, 운영 실패

Corresponding author: Soonhoi Ha (e-mail: sha@snu.ac.kr).

replay, latency, regression risk를 결합한 다목적 적합도와 regression-aware acceptance gate를 제안한다.
- Phi-3.5-mini-instruct, Qwen2.5-Coder-7B-Instruct, Llama-3.1-8B-Instruct, Gemma-2-9B-it, Qwen2.5-Coder-14B-Instruct로 구성된 size-tiered representative local model suite를 정의하고, cloud-authored prompt direct transfer baseline과 비교하여 local adaptation의 실효성을 보인다.
- JOILang 중심 실험 외에 auxiliary structured generation task를 추가하여, block-level search와 deterministic fitness의 방법 구조가 JOILang 밖 structured generation setting에서도 제한적으로 전이 가능한지 점검한다.
- 프롬프트 설계를 search–evaluation–acceptance–deployment–replay로 이어지는 PromptOps lifecycle로 재해석하고, replay-driven improvement와 schema/model drift 대응 방향을 논의한다.

2 관련 연구

본 논문과 직접 연결되는 선행축은 크게 네 가지다. 첫째는 automatic prompt optimization, 둘째는 feedback-driven refinement, 셋째는 constraint-guided generation 및 deterministic evaluation, 넷째는 PromptOps/LLMOps 계열 운영 연구이다. 이 축들은 서로 보완적이지만, block-structured prompt artifact를 local model adaptation 관점에서 evolutionary search의 대상으로 보고, replay와 deterministic validation을 search-time fitness로 통합하는 사례는 상대적으로 드물다.

2.1 자동 프롬프트 최적화와 search-based prompting

프롬프트를 최적화 가능한 프로그램으로 취급하는 흐름은 점차 강화되고 있다. APE, OPRO, EvoPrompt, Promptbreeder류 연구는 LLM 자체를 optimizer로 사용하거나, genetic/search-based 변형을 통해 수작업 prompt engineering을 대체할 수 있음을 보였다. 그러나 이들 연구는 대체로 범용 텍스트 태스크를 대상으로 하며, token 또는 sentence 수준의 변형을 중심으로 설계되고, 최종 목적함수도 개발 셋 정확도에 치우치는 경우가 많다. 반면 본 연구는 search unit을 token이 아닌 *block*으로 정의하고, local deployment 환경에서 중요한 prompt length, latency, replay recovery, deterministic correctness를 동등한 설계 목표로 포함한다.

2.2 피드백 기반 정련과 자기 수정

Self-Refine, Reflexion 계열, preference-guided optimization 연구들은 self-critique나 explicit human feedback를 이용해 출력을 반복 개선한다. 이들은 refinement의 유용성을 잘 보여주지만, 대체로 피드백이 명시적이고 curated되어 있으며, 운영 로그 자체를 구조화된 학습/탐색 신호로 활용하지는 않는다. 본 연구는 Slack 기반 운영 로그, 관리자 수정 기록, schema violation, owner mismatch와 같은 *implicit operational feedback*를 replay set으로 변환하고, 이를 search-time fitness의 핵심 구성요소로 사용한다는 점에서 차별화된다.

2.3 제약 기반 생성과 결정적 평가

구조적 DSL 생성에서는 단순 텍스트 유사도만으로 신뢰성을 보장하기 어렵다. PICARD와 grammar-guided generation은 token-level 또는 incremental parsing 기반 제약을 사용하여 잘못된 출력을 줄인다. 그러나 이러한 방식은 상용 API 기반 black-box inference나 제한된 local runtime에서는 항상 동일하게 적용되기 어렵다. 본 연구는 prompt structure와 post-generation deterministic checks를 결합하여 invalid candidate를 조기에 탈락시키는 전략을 사용한다. 또한 AST structural similarity는 true tree edit distance가 아니라 recursive structural comparison 또는 rule-based structural alignment에 가깝다고 보수적으로 기술하며, Z3를 쓰더라도 full program equivalence가 아니라 condition-level logical checking 범위로 한정한다.

2.4 PromptOps와 운영 생명주기

PromptOps 및 LLMOps 계열은 prompt versioning, CI/CD, cost tracking, governance, deployment metadata 관리 측면에서 빠르게 성장하고 있다. 그러나 이들 시스템의 많은 부분은 prompt artifact를 *관리(manage)*하는 데 초점이 있고, prompt를 search와 replay 기반으로 *자동 개선(optimize)*하는 루프까지 내장한 경우는 제한적이다. 본 연구는 replay-driven search, regression guard, acceptance gate, versioned deployment artifact를 하나의 폐루프로 묶음으로써 prompt engineering을 수작업 문구 조정이 아니라 지속적 artifact optimization lifecycle로 재구성한다.

표 1에서 보듯이, 본 논문은 개별 축을 처음 제안하는 것이 아니라, *block-level searchable prompt artifact*, *local-model adaptation*, *replay-driven feedback*, *deterministic search-time fitness*, *PromptOps deployment*를 하나의 루프로 통합하는 데 차별점이 있다. 특히 기존 reliable JOILang pipeline이 target DSL과 deterministic validation layer를 제공한다면, 본 논문은 그 위에서 prompt artifact 자체를 search 대상과 deployment unit으로 재정의한다.

3 방법론

3.1 문제 정의

입력은 자연어 명령 집합 X, 서비스 스키마 S, benchmark set B, 운영 실패로부터 구성된 replay set R, 그리고 prompt block library $P = \{b_1, \ldots, b_K\}$로 정의한다. 출력은 model weight update 없이 가장 높은 운영 적합도를 보이는 prompt artifact p^*이다. 여기서 운영 적합도는 단일 accuracy 값이 아니라 deterministic correctness, replay recovery, token budget, latency, regression safety를 함께 반영한다.

이 최적화는 모델 학습 문제가 아니라 artifact search 문제다. 즉, 본 논문의 산출물은 fine-tuned model이 아니라 accepted prompt version과 그 deployment metadata이다. 이 점은 cloud에서 잘 작동한 prompt가 local 모델에서도 그대로 최적일 것이라는 가정이 성립하지 않는 상황에서 특히 중요하다.

3.2 Block-structured Genotype

프롬프트를 검색 가능한 유전자형 g로 정의한다. 본 논문은 모든 블록이 동일한 수준의 가변성을 가진다고 가정하지 않고, 다음 세 계층으로 구성한다.

- **Constraint Blocks:** `System_Role`, `Output_Schema`, `Safety_Guard`, `Temporal_Rule`. 이들은 형식과 안전성 일관성을 강하게 유지하는 블록이다. 특히 `Output_Schema`는 `device: { info: ..., examples: ... }` 형태의 중첩 딕셔너리 구조를 강제한다.
- **Adaptive Blocks:** `Skeleton-of-Thought`, `Service_Mapping`, `Task_Definition`, `Anti_Hallucination`, `Owner_Device_Rule`. 이들은 모델 family, 질의 유형, schema 상태에 따라 wording 또는 activation이 달라질 수 있다.

TABLE 1
관련 연구 축별 비교. 각 열은 본 절에서 논의한 대표 연구 흐름을 나타내며, 마지막 열은 본 논문의 위치를 요약한다.

	Search-based prompt optimization [1], [2], [3]	Feedback-driven refinement [4], [5]	Constraint-guided decoding & deterministic checks [6], [7]	PromptOps / LLMOps [8], [9]	Reliable JOILang pipeline [10]	**Ours**
Block-level searchable prompt artifact	limited	–	–	partial	partial	✓
Local-model adaptation	limited	limited	–	partial	limited	✓
Replay-driven operational feedback	–	partial	–	partial	partial	✓
Deterministic evaluation as search-time fitness	–	–	partial	–	partial	✓
Token / latency-aware optimization	partial	limited	limited	partial	–	✓
Regression-aware acceptance gate	–	–	partial	partial	partial	✓
Versioned prompt / config deployment	–	–	–	✓	–	✓
JOILang / DSL-specific generation	–	–	partial	–	✓	✓

✓: 명시적으로 지원; partial: 부분적 또는 간접적으로 지원; limited: 제한적으로만 고려; –: 핵심 설계 목표 아님.

TABLE 2
Prompt block taxonomy used in the genotype design

Block type	**Example blocks**	**Primary role**
Constraint	system_role, output_schema, safety_guard, temporal_rule	형식, 정책, 안전 제약 유지
Adaptive	task_definition, service_mapping, anti_hallucination, owner_device_rule	모델/질의 유형별 wording 조정
Conditional	few_shot, repair_after_regression, corner_case_repair, retrieval_injection_policy	replay/drift/retrieval 조건별 보강

- **Conditional Blocks:** `Few_Shot`, `Version_Guard`, `Repair_After_Regression`, `Corner_Case_Repair`, `Retrieval_Injection_Policy`. 이들은 retrieval context, replay category, drift 상태에 따라 조건부로 삽입된다.

유전자형은 다음과 같이 표현할 수 있다.

$$g = \{a_k, o, w_k, f, r, c, q, v\}, \tag{1}$$

여기서 a_k는 각 block의 활성화 여부, o는 block 순서, w_k는 wording template 선택, f는 few-shot choice, r은 retrieval mode, c는 compression level, q는 replay-repair toggle, v는 regression-guard toggle을 의미한다. 표현형 $p(g)$는 활성 블록과 선택된 wording, few-shot, retrieval-conditioned context를 순서대로 조립한 실제 prompt이다.

이 표현의 장점은 세 가지다. 첫째, token-level mutation과 달리 기능적으로 해석 가능한 단위에서 탐색이 이루어져 search space가 구조화된다. 둘째, 운영자가 acceptance 이후 어떤 block family가 개선 또는 regression에 기여했는지 추적하기 쉽다. 셋째, retrieval-conditioned assembly를 허용하므로 질의 유형과 검색 문맥에 따라 prompt 구성이 유연하게 조정된다.

3.3 Evolutionary Prompt Search

초기 population은 현재 운영 prompt, hand-crafted best prompt, minimal fixed-block prompt, retrieval-heavy prompt, token-compressed prompt, model-specific seed, replay-repair seed로 구성한다. 선택(selection)은 elitism과 tournament selection을 조합하며, regression-causing individual에는 penalty를 부여한다. crossover는 block-level recombination, mutation은 block insertion/deletion, activation toggle, reorder swap, wording mutation, few-shot replacement, retrieval mode switch, compression mutation, replay clause insertion, regression-guard strengthening/weakening으로 설계한다.

대표적인 fitness는 다음과 같이 쓸 수 있다.

$$\begin{aligned} f(g) = \alpha \cdot \text{DETPass} + \beta \cdot \text{SDET} + \gamma \cdot \text{ReplayAcc} + \delta \cdot \text{CategoryBalance} \\ - \lambda_1 \cdot \text{Tokens} - \lambda_2 \cdot \text{Latency} - \lambda_3 \cdot \text{Cost} - \lambda_4 \cdot \text{RegressPenalty}. \end{aligned} \tag{2}$$

여기서 DETPass는 schema, parser, static rule, policy constraint를 모두 통과한 비율이고, SDET는 정답 트리와의 구조적 유사도, ReplayAcc는 운영 실패 replay에 대한 회복률을 의미한다. 이 식은 최종 고정 수식이라기보다 *one operational instantiation*이며, 계수는 deployment priority에 따라 calibration 대상이 된다.

3.4 Full B6 Configuration and Regression Guard

본 논문에서 **B6 (Full GPS / PromptOps)**는 benchmark split과 replay split을 동시에 활용하는 full search configuration이다. 각 candidate prompt는 먼저 deterministic validation

TABLE 3
Fitness components and their roles in search and acceptance

Component	Fitness	Gate	Primary role
DETPass	✓	✓	형식 및 정책 유효성 보장
SDET	✓	–	구조적 근접도 보완
ReplayAcc	✓	✓	운영 실패 복구 평가
SchemaRobustness	✓	partial	drift/noisy context 강건성
Tokens / Latency / Cost	✓	–	배포 효율성 반영
RegressPenalty	✓	✓	평균 개선 대비 핵심 subset 회귀 방지

layer를 통해 schema, parser, policy, temporal checks를 평가받고, 이후 DETPass, structural similarity, replay recovery, token length, latency, regression penalty를 포함한 multi-objective fitness로 ranking된다. Semantic judge는 모든 candidate에 일괄 적용하지 않고, deterministic score 상위의 ambiguous top-k 후보에만 제한적으로 적용된다. 최종 acceptance 단계에서는 overall DETPass 하한, replay accuracy 하한, critical subset non-regression 조건을 동시에 만족하는 후보만 versioned prompt artifact로 채택한다.

여기서 중요한 설계 요소는 **regression guard**이다. regression guard는 새 prompt candidate가 평균 성능을 일부 개선하더라도, 기존 accepted prompt 대비 safety-critical subset, temporal subset, replay-critical subset, 또는 schema-validity 지표에서 의미 있는 성능 저하를 보일 경우 채택을 제한하는 메커니즘이다. 구현상으로는 search-time fitness에서 soft penalty로 반영하고, acceptance 단계에서는 핵심 subset에 대한 hard non-regression constraint를 추가로 적용한다.

search-time soft penalty는 다음과 같이 둘 수 있다.

$$\text{RegressPenalty}(g) = \sum_{c \in \mathcal{C}} w_c \cdot \max\left(0,\, m_c(p_{\text{prev}}) - m_c(g) - \epsilon_c\right), \tag{3}$$

여기서 p_{prev}는 현재 운영 중인 accepted prompt, $m_c(\cdot)$는 category/subset c에서의 metric, ϵ_c는 허용 가능한 작은 변동폭을 의미한다.

acceptance 단계의 hard non-regression constraint는 다음과 같이 표현할 수 있다.

$$\text{DETPass}(g) \geq \tau_{\text{det}}, \tag{4}$$

$$\text{ReplayAcc}(g) \geq \tau_{\text{rep}}, \tag{5}$$

$$m_c(g) \geq m_c(p_{\text{prev}}) - \epsilon_c, \quad \forall c \in \mathcal{C}_{\text{critical}}, \tag{6}$$

$$\text{SchemaFailRate}(g) \leq \text{SchemaFailRate}(p_{\text{prev}}) + \delta. \tag{7}$$

이 정의는 regression guard를 단순한 개념 용어가 아니라, search-time penalty와 deployment-time acceptance rule로 분해된 조작적 메커니즘으로 만든다는 점에서 중요하다.

3.5 PromptOps와 Replay Integration

본 파이프라인은 일회성 탐색이 아니라 운영 피드백 기반의 폐루프를 가진다. 자동화 Slack 봇 연동이나 관리자 운영 과정에서 발생하는 `user_name` 식별 오류, owner mismatch, schema violation, temporal misinterpretation, formatting failure 등의 사례는 **Replay Set**으로 구조화된다. 이후 GA 탐색은 benchmark와 replay를 동시에 활용하여 candidate prompt를 생성하고, deterministic evaluation과 선택적 semantic judge로 이를 평가한다.

수용된 prompt는 version scaffold를 생성하고 demo/web 또는 실제 service deployment로 이동한다. 이후 운영 중 새 실패 케이스가 다시 누적되면 다음 세대 탐색의 replay signal이 된다. 이로써 prompt engineering은 search–evaluation–acceptance–deployment–replay로 이어지는 PromptOps lifecycle이 된다.

3.6 Deterministic and Hybrid Evaluation

Deterministic evaluation은 prior JOILang work의 기반 계층을 재사용하되, 본 연구에서는 역할을 post-hoc metric에서 *search-time fitness signal*로 확장한다. 검사 항목은 schema check, syntax/parser check, static rule check, service/policy constraint check, temporal construct check, AST structural similarity를 포함한다.

표현은 보수적으로 유지해야 한다. AST structural similarity는 true tree edit distance가 아니라 recursive structural comparison 또는 rule-based structural alignment에 가깝다. 또한 Z3를 사용하더라도 full program equivalence가 아니라 if-clause나 predicate 수준의 *condition-level logical checking*으로 제한하여 기술한다.

Semantic judge는 DET-only layer가 놓칠 수 있는 intent mismatch를 보완하는 선택적 보조 신호로 사용한다. 이 judge는 모든 후보에 일괄 호출하기보다 상위 후보 또는 ambiguous case에만 호출하는 것이 실무적으로 적절하다. 따라서 본 논문은 judge를 deterministic layer의 대체물이 아니라 assistive component로만 사용하며, 후속 실험에서는 DET-only, LLM-only, Hybrid Judge를 비교하고 human agreement를 통해 신뢰도를 평가한다.

4 실험 설정

4.1 Representative Local Model Suite

본 논문은 local deployment를 대표하기 위해 small, mid, upper-mid 규모를 아우르는 size-tiered local model suite를 사용한다. 구체적으로, Phi-3.5-mini-instruct를 초경량 baseline으로, Qwen2.5-Coder-7B-Instruct를 search-efficient code-centric local model로, Llama-3.1-8B-Instruct와 Gemma-2-9B-it를 family/instruction-style diversity 확보를 위한 mid-tier 비교군으로, 그리고 Qwen2.5-Coder-14B-Instruct를 single high-end workstation-class GPU 또는 quantized local inference 환경에서의 deployment-oriented target으로 선택하였다. 이 구성은 (i) open-weight 또는 local 실행 가능성, (ii) structured code generation 적합성, (iii) family diversity, (iv) small-model portability와 upper-mid deployment practicality를 동시에 반영한다. Cloud 모델은 최적화 대상이 아니라 strong prompt의 출발점으로 사용되며, cloud-authored prompt direct transfer baseline을 제공한다.

4.2 Dataset and Evaluation Sets

데이터셋은 세 층으로 구성된다. **Benchmark Set**은 JOICommands-170 기반의 표준 비교용 데이터, **Replay Set**은 운영 실패 또는 관리자 수정 이력을 구조화한 데이터, **Stress Set**은 schema drift, ambiguity, long-context, temporal complexity를 포함한 확장 평가용 데이터다. 본 논문에서는 Benchmark Set 170개, Replay Set 50개, Stress Set 30개를 사용한다. 또한 JOILang 내부 성능 향상을 방법론 일반성 주장으로 과장하지 않기 위해, schema-constrained function-call/JSON generation 형태의 **Auxiliary Structured Set** 120개를 추가로 사용한다.

TABLE 4
Size-tiered representative local model suite and selection rationale

Model	Tier	Role	Selection rationale
Phi-3.5-mini-instruct	Small	Efficiency floor	초경량 local deployment와 GA 탐색 비용 절감을 위한 small baseline
Qwen2.5-Coder-7B-Instruct	Small-mid	Search anchor	structured code generation에 적합한 code-centric 7B 모델, Qwen family 내 scale-up 비교의 anchor
Llama-3.1-8B-Instruct	Mid	Family diversity	prompt portability와 instruction-style 차이를 점검하기 위한 대표 8B 모델
Gemma-2-9B-it	Mid	Style diversity	instruction 스타일과 structured output 안정성 차이를 반영하는 mid-tier 비교군
Qwen2.5-Coder-14B-Instruct	Upper-mid	Deployment target	단일 고성능 워크스테이션급 GPU 또는 quantized inference 환경에서 실배포를 겨냥한 upper-mid code model
GPT-4.1-mini	Cloud ref.	Upper reference	cloud-authored prompt direct transfer baseline과 상위 성능 기준선 제공

카테고리는 simple one-shot, conditional control, temporal flow, multi-device coordination, safety-constrained command, ambiguous command, long-context command, schema-shifted command, replayed failure case로 나눈다.

4.3 Baselines and Ablations

핵심 baseline은 다음과 같다.

- **B0 (Hand-crafted):** 전문가가 직접 설계한 현재 운영 prompt.
- **B1 (Cloud Reference):** GPT-4.1-mini와 무거운 few-shot prompt.
- **B2 (Cloud-to-Local Transfer):** B1의 cloud-authored prompt를 local 모델에 그대로 주입.
- **B3 (Fixed Block):** block 구조는 유지하되 탐색 없이 고정된 조합을 사용하는 방식.
- **B4 (Random Search):** block space에서 무작위 탐색.
- **B5 (GA + Benchmark Only):** replay 없이 benchmark 성능만으로 탐색.
- **B6 (Full GPS / PromptOps):** benchmark, replay, deterministic validation, regression guard를 모두 포함한 제안 방식.

또한 **No replay**, **No deterministic fitness**, **No semantic judge**, **No regression guard**, **No crossover** ablation을 포함하여 각 단계의 정당성을 검증한다.

4.4 Auxiliary Structured Generation Task

Auxiliary task는 JOILang 밖 structured generation setting에서 block-level search와 deterministic fitness, regression-aware acceptance가 제한적으로 유효한지 점검하기 위해 추가되었다. 이 실험의 목적은 새로운 state-of-the-art를 주장하는 데 있지 않고, 방법 구조의 전이 가능성을 최소 수준에서 확인하는 데 있다. 이를 위해 schema-constrained function-call/JSON argument generation 문제를 사용하며, Qwen2.5-Coder-7B-Instruct와 Qwen2.5-Coder-14B-Instruct를 대상으로 B0, B2, B3, B6를 비교한다. 평가 지표는 schema pass rate, task-level semantic correctness, average input tokens, latency를 포함한다.

5 결과 및 분석

표 7는 cloud-authored prompt를 local 모델에 직접 전이했을 때와, 제안하는 GPS/PromptOps 탐색을 적용했을 때의 차이를 보여준다. 직접 전이(B2)는 모든 local 모델에서 입력 토큰 수를 3,500개 수준까지 증가시키지만, DETPass와 ReplayAcc 개선은 제한적이거나 오히려 불안정하다. 이는 cloud model을 위해 설계된 prompt가 local model family의 instruction density와 formatting preference에 맞지 않음을 의미한다.

반면 제안 방식(B6)은 다섯 개 local 모델 평균 기준으로 B2 대비 DETPass를 5.9%p, ReplayAcc를 13.1%p 향상시키면서도 입력 토큰 수를 66.7% 감소시킨다. 특히 Qwen2.5-Coder-14B에서는 GPT-4.1-mini의 cloud reference에 근접한 94.0% DETPass를 달성했고, Qwen2.5-Coder-7B에서도 strong local baseline으로서 92.6% DETPass를 보였다. Phi-3.5-mini와 같은 경량 모델에서도 direct transfer 대비 큰 폭의 recovery가 관찰되며, 이는 small-tier부터 upper-mid deployment tier까지 prompt artifact search의 실질적 가치를 뒷받침한다.

표 8는 deployment-oriented representative model인 Qwen2.5-Coder-14B에서 각 구성요소의 기여를 분해한다. Replay를 제거하면 평균 정확도 자체보다 운영 실패 복구 성능이 더 크게 하락하며, deterministic fitness를 제거하면 semantic score보다 형식 · 정책 위반이 먼저 증가한다. Regression guard를 제거했을 때 overall 평균 점수는 크게 유지될 수 있지만, critical subset에서의 regression rate가 눈에 띄게 증가한다. 이는 regression guard가 단순 평균 향상보다 deployment acceptance 안정성에 더 직접적으로 기여함을 시사한다.

표 9는 benchmark-only search와 replay-aware search의 차이를 더 직접적으로 보여준다. Replay-aware B6는 replay failure recovery뿐 아니라 schema drift와 long-context를 포함한 stress setting에서도 더 높은 DETPass를 유지한다. 특히 regression rate의 감소는 replay와 regression guard의 결합이 deployment 안정성 측면에서 유의미함을 보여준다.

표 10는 JOILang 밖 structured generation setting에서의 제한적 전이 가능성을 보여준다. B6는 auxiliary task에서도 direct transfer보다 높은 schema pass와 semantic correctness를 보이며, 동시에 더 짧은 입력으로 동작한다. 그러나 이 실험은 broad generalization을 주장하기 위한 것이 아니라, block-level search와 deterministic fitness가 특정 structured generation setting에 부분적으로 이식될 수 있음을 점검하기 위한 보조 증거로 해석해야 한다.

Judge reliability 측면에서는 Hybrid Judge가 가장 높은 agreement와 rank stability를 보인다. 이는 deterministic layer가 명백한 형식/정책 위반을 먼저 걸러내고, semantic judge가 잔여 ambiguous case를 보완하는 구조가 단일 judge보다 안정적임을 시사한다. 다만 이 수치는 human agreement 실험이 충분히 확정된 이후 다시 검증되어야 하며, semantic judge를 인간과 동등한 최종 판정기로 해석해서는 안 된다.

5.1 해석과 시사점

본 논문에서 관찰된 핵심 패턴은 다섯 가지다. 첫째, cloud direct transfer는 local 모델별 prompt portability 한계를 명확히 드러낸다. 둘째, fixed or hand-crafted prompt는 baseline으로 유효하지만, block-level evolutionary search를 붙였을 때 accuracy–token Pareto가 더 개선된다. 셋째, replay-aware search는 benchmark-only search보다 운영 실패 복구와 regression control에서 더 실용적이다. 넷째, improvement의 크기는 model-specific하며, 하나의 universal prompt가 모든 local 모델에 최적이라는 식의 일반화는 성립하지 않는다. 다섯째, auxiliary structured task 결과는 본 방법 구조가 JOILang

TABLE 5
Dataset composition and evaluation splits

Set	Purpose	#Samples	Avg NL Len.	Avg Target Len.	Temporal Ratio	Safety Ratio	Special
Benchmark	표준 성능 비교	170	18.6	8.1	0.21	0.18	category-balanced
Replay	운영 실패 복구 평가	50	20.9	8.7	0.24	0.20	Slack/admin fai
Stress	drift/ambiguity 평가	30	35.8	10.1	0.30	0.27	schema shift, long c
Auxiliary	방법 구조 전이 점검	120	16.8	5.4	0.12	0.09	schema-constrained functi

TABLE 6
Core baselines and the reviewer concern each baseline addresses

Variant	Primary role
Hand-crafted prompt	수작업 기준선과의 직접 비교
Cloud direct transfer	local adaptation necessity 검증
Fixed block	blockization 자체와 search advantage 분리
Random search	search가 trivial mutation이 아님을 확인
GA + benchmark only	replay signal의 추가 가치 검증
Full GPS / PromptOps	제안법의 종합 효과 측정

밖 structured generation에도 일정 부분 이식될 수 있음을 시사하지만, 그 범위는 제한적으로 해석되어야 한다.

6 토의

본 논문이 제시하는 가장 중요한 관점은 prompt engineering을 versioned artifact optimization 문제로 바꾸는 것이다. 이 framing은 local model proliferation과 deployment constraint가 강한 환경에서 특히 유용하다. weight tuning은 비용과 운영 부담이 크지만, prompt artifact는 빠르게 수정, 평가, 배포할 수 있기 때문이다.

Block-level search가 유리한 이유는 search space를 의미 단위로 구조화할 수 있기 때문이다. `system role`, `temporal rule`, `safety guard`, `few-shot`과 같은 단위는 기능적으로 해석 가능하므로, 왜 특정 변형이 개선이나 regression을 만들었는지 추적하기 쉽다. 이는 PromptOps 관점에서 감사 가능성과 버전 관리 용이성을 동시에 높인다.

Replay-aware optimization의 중요성도 크다. Benchmark-only optimization은 평균 성능을 높일 수 있지만, 실제 운영에서 반복적으로 발생한 failure mode를 놓칠 수 있다. 반대로 replay set은 배포 후 실패를 다음 세대 search의 신호로 재주입함으로써 실전 적합도를 높인다. 이는 accuracy-only 논문과 실서비스형 논문을 가르는 중요한 차이이다. 특히 regression guard가 soft penalty와 hard gate로 함께 동작할 때, average metric 개선이 critical subset의 regression을 가리는 문제를 줄일 수 있다.

동시에 auxiliary structured generation task는 본 연구가 JOILang 안에서만 성립하는 완전히 폐쇄적인 engineering trick이 아니라는 점을 부분적으로 지지한다. 그러나 이 보조 실험은 어디까지나 방법 구조의 전이 가능성을 제한적으로 보여주는 수준이며, 일반 목적 structured generation이나 임의의 DSL에 대한 보편적 우수성을 입증하는 것으로 해석해서는 안 된다.

7 한계

본 논문의 주된 실험 증거는 JOILang domain에 집중되어 있으며, auxiliary structured generation task는 방법 구조의 제한적 전이 가능성만을 보여준다. 따라서 본 연구는 general-purpose prompt optimization에 대한 보편 해법을 주장하지 않는다. 또한 semantic judge는 assistive component일 뿐이며 human-equivalent oracle이 아니다. Drift 실험 역시 controlled schema-shift setting에 국한되며, 실서비스의 장기적 distribution shift 전체를 포괄하지 않는다. 마지막으로 본 논문은 스마트홈/스마트오피스 수준의 service-level automation을 주된 범위로 두며, 산업용 safety-critical deployment에 대한 직접적 일반화는 future work로 남긴다.

8 결론

본 논문은 JOILang이나 기존 reliability pipeline을 재제안하지 않고, JOILang generation을 위한 prompt artifact optimization 문제를 중심에 둔다. 핵심은 block-structured genotype, GA/evolutionary search, benchmark+replay+deterministic fitness, regression-aware acceptance gate, 그리고 PromptOps lifecycle의 통합이다.

Representative local model suite를 기준으로 보면, 본 논문의 가치는 한 가지 universal prompt를 주장하는 데 있지 않다. 오히려 small-tier부터 upper-mid deployment tier까지 local 모델별로 prompt density, wording, few-shot, retrieval-conditioned assembly가 다르게 최적화되어야 한다는 점을 evidence-first 방식으로 보여주는 데 있다. 또한 auxiliary structured generation task는 본 방법 구조가 JOILang 밖 structured generation setting에도 제한적으로 전이될 수 있음을 보조적으로 시사한다.

향후 과제로는 schema drift에 대한 warm-start re-search, cross-model transferability의 정량 분석, 더 경량한 semantic judge, 운영 로그 기반 replay taxonomy의 표준화, 그리고 auxiliary task의 다양화를 통한 방법 구조 검증이 있다. 그럼에도 본 연구는 prompt engineering을 수작업 문구 조정이 아니라 search, evaluation, acceptance, deployment로 이어지는 PromptOps lifecycle 문제로 정식화했다는 점에서, local LLM 기반 IoT 자동화를 위한 실무적 토대를 제공한다.

References

[1] D. Zhou *et al.*, "Large language models are human-level prompt engineers," *arXiv preprint*, 2023.

[2] C. Yang *et al.*, "Large language models as optimizers," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[3] Q. Guo *et al.*, "Connecting large language models with evolutionary algorithms yields powerful prompt optimizers," *arXiv preprint*, 2023.

[4] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," *arXiv preprint*, 2023.

[5] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, 2023.

[6] T. Scholak, N. Schucher, and D. Bahdanau, "Picard: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 9895–9901, 2021.

TABLE 7
Main results: local-model adaptation under cloud-reference prompting

Model	Variant	DETPass (%)	Mean SDET	ReplayAcc (%)	Avg Input Tokens	Latency (s)
GPT-4.1-mini (Cloud)	B1 (Cloud reference)	94.5	0.89	92.0	3,500	2.85
Phi-3.5-mini	B0 (Hand-crafted)	80.8	0.66	72.1	1,450	1.21
	B2 (Cloud transfer)	78.6	0.65	71.0	3,500	1.92
	B6 (Full GPS / Ours)	**86.1**	**0.79**	**86.9**	**980**	**1.00**
Qwen2.5-Coder-7B	B0 (Hand-crafted)	86.4	0.72	78.4	1,450	1.47
	B2 (Cloud transfer)	87.8	0.73	80.8	3,500	2.28
	B6 (Full GPS / Ours)	**92.6**	**0.87**	**91.2**	**1,120**	**1.25**
Llama-3.1-8B	B0 (Hand-crafted)	82.9	0.69	74.6	1,450	1.62
	B2 (Cloud transfer)	81.7	0.68	72.4	3,500	2.46
	B6 (Full GPS / Ours)	**88.7**	**0.82**	**89.4**	**1,250**	**1.40**
Gemma-2-9B	B0 (Hand-crafted)	84.2	0.70	76.5	1,450	1.66
	B2 (Cloud transfer)	83.3	0.69	75.8	3,500	2.52
	B6 (Full GPS / Ours)	**89.4**	**0.84**	**90.1**	**1,180**	**1.44**
Qwen2.5-Coder-14B	B0 (Hand-crafted)	88.9	0.75	82.1	1,520	1.89
	B2 (Cloud transfer)	90.1	0.77	84.3	3,500	2.88
	B6 (Full GPS / Ours)	**94.0**	**0.89**	**92.2**	**1,290**	**1.63**

TABLE 8
Ablation results on Qwen2.5-Coder-14B as a deployment-oriented representative local model

Variant	DETPass	SDET	ReplayAcc	Tokens
B6 Full GPS	94.0	0.89	92.2	1,290
B3 Fixed block	90.7	0.80	84.8	1,360
B4 Random search	90.1	0.79	84.2	1,350
B5 GA + benchmark only	92.4	0.86	87.1	1,276
No replay	92.2	0.86	86.4	1,275
No semantic judge	93.1	0.87	90.1	1,268
No deterministic fitness	90.9	0.82	87.5	1,280
No regression guard	92.5	0.85	88.6	1,286
No crossover	92.8	0.85	88.9	1,298

TABLE 9
Replay and drift recovery on representative Qwen models

Model	Variant	ReplayAcc	Stress DETPass	Regress. Rate
Qwen2.5-Coder-7B	B2 Cloud transfer	80.8	74.2	14.6
Qwen2.5-Coder-7B	B5 Benchmark only	86.5	82.1	9.2
Qwen2.5-Coder-7B	B6 Full GPS	91.2	87.3	4.1
Qwen2.5-Coder-14B	B2 Cloud transfer	84.3	78.5	12.0
Qwen2.5-Coder-14B	B5 Benchmark only	89.1	86.7	7.8
Qwen2.5-Coder-14B	B6 Full GPS	92.2	90.4	2.9

TABLE 10
Auxiliary structured generation task: limited transfer evidence of the method structure

Model	Variant	Schema Pass	Semantic Corr.	Tokens	Regress. Rate
Qwen2.5-Coder-7B	B0 Hand-crafted	88.2	82.4	980	10.7
Qwen2.5-Coder-7B	B2 Cloud transfer	85.9	79.7	2,400	11.9
Qwen2.5-Coder-7B	B3 Fixed block	87.5	81.3	1,020	7.8
Qwen2.5-Coder-7B	B6 Full GPS	90.4	84.9	760	4.9
Qwen2.5-Coder-14B	B0 Hand-crafted	89.7	84.0	1,020	9.6
Qwen2.5-Coder-14B	B2 Cloud transfer	86.7	80.1	2,400	8.7
Qwen2.5-Coder-14B	B3 Fixed block	90.1	84.8	1,040	6.4
Qwen2.5-Coder-14B	B6 Full GPS	93.1	88.4	820	2.9

TABLE 11
Judge reliability plan and pilot settings

Judge setting	Agreement	Cohen's κ	Rank stability
DET-only	0.81	0.62	medium
LLM-only	0.84	0.68	medium
Hybrid Judge	0.89	0.77	high

[7] S. Geng *et al.*, "Grammar-constrained decoding for structured nlp tasks without finetuning," *arXiv preprint*, 2023.

[8] To be replaced with verified PromptOps citation, "Promptops reference placeholder," 2025. Placeholder entry. Replace with a verified PromptOps or prompt lifecycle management reference before submission.

[9] To be replaced with verified LLMOps citation, "Llmops reference placeholder," 2025. Placeholder entry. Replace with a verified LLMOps / production operations reference before submission.

[10] M. Jeong, E. Jeong, and S. Ha, "Joilang: Reliable natural language-to-dsl generation for service-level iot automation." Under review / reviewed manuscript, bibliographic details to be finalized before submission, 2023.

TABLE 12
Threats to validity and scope

Threat	Why it matters	Mitigation / remaining scope
Single-domain main evidence	JOILang 특화성으로 일반화 제한	auxiliary task 추가, 그러나 broad generalization은 주장하지 않음
Judge limitation	semantic score의 black-box 우려	DET-first hybrid usage, human agreement로 보완
Controlled drift only	실제 장기 drift와 다를 수 있음	stress set으로 제한적 점검, full production drift는 future work
Hardware-dependent latency	환경에 따라 latency가 변함	동일 환경 비교, absolute latency 일반화 금지
Model-specific prompts	cross-model transferability 저하 가능	family diversity 실험 포함, universal prompt는 주장하지 않음